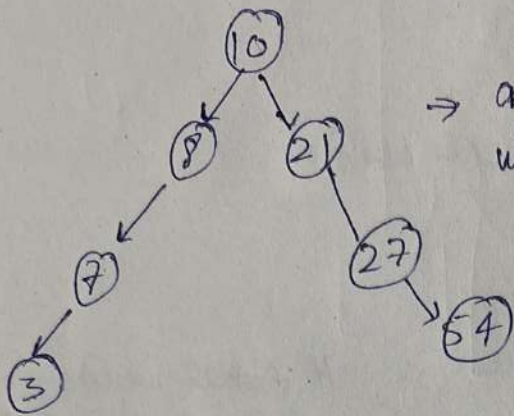


INSERTION IN BST

[10, 8, 21, 7, 27, 5, 4, 3]



→ agr chota hai to left mai jata hai
warna right mai jata

algo →
⇒ if koi number aya, use
compare kara (root → val) se

agr chota hua to left, else right mai jaege

```
class Node {  
    public;  
    int data;  
    Node* left;  
    Node* right;
```

```
Node (data)  
    this → data = data;  
    this → right = this → left = NULL;
```

```
public:
```

```
void insert(Node* root, int data)
```

```
{
```

```
    // base case (root == NULL) < (1st data)
```

```
    root = new Node(d);  
    return root;
```

```
}
```

```
    if (data > root → val)
```

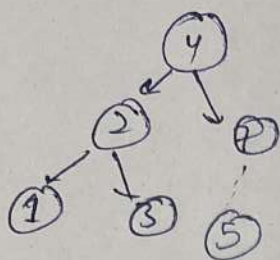
```
    root → right = insert(root → right, data);    // if data > R → val
```

```
    else {
```

```
        root → left = insert(root → left, data);    // if data < R → val;
```

```
    }
```


LC-701 → (Insert into a BST)



⑤ to add;

// we'll keep track & keep changing the location by comparing the left & right.

if (root == Null)

return new TreeNode(val);

if (val >= root->val)

<

root->right = insert(root->right, val);

else <

root->left = insert(root->left, val);

return root;

In this question we have to insert in a BST;

to insert in a BST, we have to check for the conditions i.e. if the val provided is smaller than the (root->val) we will insert it into the (left subtree)

else < we will be inserting it into the (right subtree);

Search a node in BST

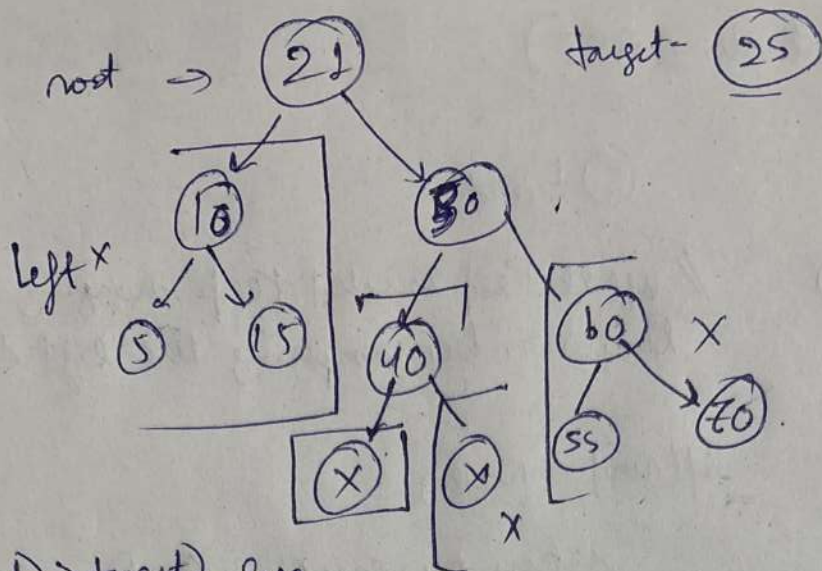
Search(root, val)

< if (root == Null) return;

if (root->val == target) return true;

if (root->val > target) return search(root->right, val);

else < return search(root->left, val);



if (root $\rightarrow$ val) > target) left
 else $\rightarrow$ to right!

TC $\rightarrow O(\log n)$

Worst case $\rightarrow O(n)$

```

if (root == Null) return false;
if (root  $\rightarrow$  val == target) return true;
if (root  $\rightarrow$  data > x) return search (root  $\rightarrow$  right, val);
else  $\rightarrow$  return search (root  $\rightarrow$  left, val);
  
```

Iterative : Binary tree $\text{temp} = \text{root};$
 while (!root)

$\leftarrow$ if (root $\rightarrow$ data == val) return true;

if (root $\rightarrow$ data > val)

$\leftarrow$ root = root $\rightarrow$ right;

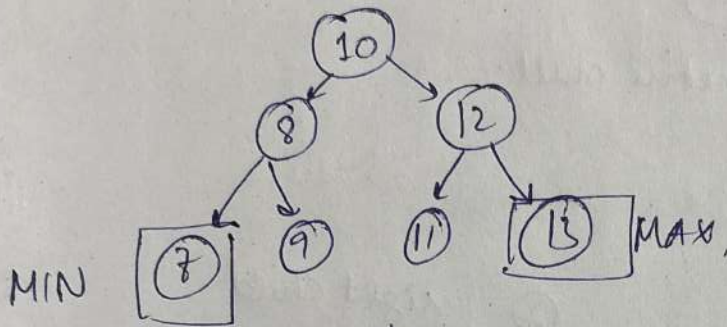
$\leftarrow$

else $\leftarrow$ root = root $\rightarrow$ left;

$\leftarrow$ return false; // not found

Minimum Value of a BST

As we know, the left subtree always have smaller value than the root $\rightarrow$ val
so, we will, just iterate to the left, most cell,
else, if we want to have MAX value within the tree, we will iterate
to the right subtree, until it is NULL,



for min value:

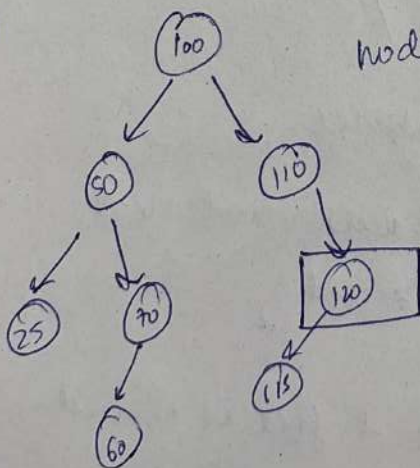
```
Node * temp = root;
while (!temp->left)
    temp = temp->left;
return temp->val;
```

for max

```
Node * temp = root;
while (!temp->right)
    temp = temp->right;
return temp->val;
```

Inorder predecessor / Successor

Deletion



node to delete $\rightarrow$ (120)

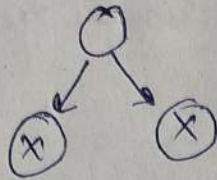
① reach to certain node:

```
if (root->data == x)
    // reached;
else if (root->data > x)
    // left;
else
    // right;
```


~~4 case prelu~~

There are 4 cases to delete a node:

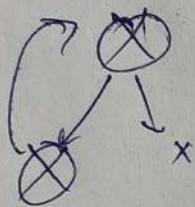
① if No child present



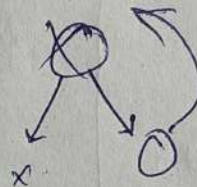
Simply delete the node and return null;

② One child

① left child



② right child



In these case simply delete the provided node & return the child node to its positions.

if $top = root \rightarrow left$:
① save
delete root
return top
② delete
③ return value

right.

④ both children are present / subtree

① left se max uthas & replace with $root \rightarrow val$;

② delete the max value we bought,

② Right mai se minimum uthas & fill it to $root \rightarrow val$;

Node* delete (root, val)

*

Base case,

(root == null) return root;

if (root->val == data)

* // 0 child;

if (root->left == null && root->right == null)

delete root;
return NULL;

}

* // 1 child

if (!root->left && root->right == null)

temp = root->left;

delete root;

return temp;

}

// same for right;

* // 2 child

if (left == null && !right)

int min = minvalue (root->right) -> data; ①

root->data = min;

root->right = delete (root->right, min); ②

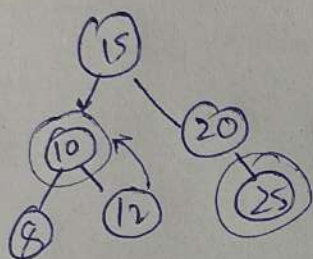
(delete right with the value we got) IMP.

INORDER PREDECESSOR

In a BST, left subtree is always smaller than the root $\rightarrow$ val whereas right subtree is always greater than the root $\rightarrow$ val, so, Inorder traversal hamesha sorted rehta hai;

ab inorder predecessor?

$\rightarrow$ Koi bhi node $\rightarrow$ just (chota element predecessor) hote hai.



INORDER $\rightarrow$ 8, 10, 12, 15, 20, 25

12 ka inorder predecessor hoga 10.

BST deletion mai predecessor ka role?

$\rightarrow$ If 2 children ho?

$\Rightarrow$ predecessor nikalo uska copy karw and swap/delete predecessor

* If a node has a left subtree, its inorder predecessor is the max element in that subtree;

INORDER predecessor $\rightarrow$ left subtree ka maximum;

INORDER successor $\rightarrow$ right subtree ka minimum;

$\Rightarrow$ In Recursion:

Recursion function ka return = update subtree ka root hote hai

$\rightarrow$ hum tree recursion mai node nahi

$\rightarrow$ naya root return karta hai.

Basic Template

```
TreeNode* solve(TreeNode* root)
```

```
if (root == NULL) return null;
```

```
root->left = solve(root->left);  
root->right = solve(root->right);
```

 } some updated subtree milta hai.

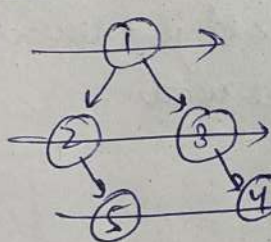
```
return root;
```

```
# root->left = deleteNode(root->left, key);
```

→ Is case mai hume updated root mila,

~~LC~~ → 199

Binary tree's Right Side View



[1, 3, 4], is problem koun agr BFS se karne do bas kar aake.

→ multitraversal mai

→ last element update krna hai,

→ or after level is done

→ for() loop end hua just return / push into

main vector;

Isme humne, BFS use kra.

```
queue<TreeNode*> q;
```

```
q.push(root);
```

```
while(!q.empty())
```

```
{ int l = 0;
```

```
int n = q.size();
```

```
for(i = 0;
```

```
TreeNode* nd = q.front();
```

```
q.pop();
```

```
le = node->val;
```

```
if (node->left)
```

```
{ q.push(node->left);
```

```
if (node->right)
```

```
{ q.push(node->right);
```

```
s.push(le);
```

return s;